你說：

題目：實作一個「骰子模擬器」。

規格：

l視窗標題「骰子模擬器」，尺寸 400×320，關閉時結束程式，開啟時置中

l中央有一個 JLabel 顯示目前點數，字體 60pt

l下方一個「擲骰子」按鈕

l每按一次隨機產生 1–6 的點數並更新顯示

l視窗上方顯示「已擲 N 次，總和 M，平均 X.XX」

l點數為 6 時文字變綠色，為 1 時變紅色，其餘黑色 請幫我生成程式碼

ChatGPT 說：

可以，這題用 **Java Swing** 很適合。下面這份程式碼可以直接執行，符合你列出的所有規格。

```
import javax.swing.*;
import java.awt.*;
import java.util.Random;

public class DiceSimulator extends JFrame {

    private JLabel diceLabel;
    private JLabel infoLabel;
    private int count = 0;  // 已擲次數
    private int sum = 0;    // 點數總和
    private Random random = new Random();

    public DiceSimulator() {

        // 視窗設定
        setTitle("骰子模擬器");
        setSize(400, 320);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // 開啟時置中

        // 使用 BorderLayout
        setLayout(new BorderLayout());

        // 上方資訊
        infoLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", JLabel.CENTER);
        infoLabel.setFont(new Font("Microsoft JhengHei", Font.PLAIN, 18));
        add(infoLabel, BorderLayout.NORTH);

        // 中央骰子點數
        diceLabel = new JLabel("0", JLabel.CENTER);
        diceLabel.setFont(new Font("Arial", Font.PLAIN, 60));
        diceLabel.setForeground(Color.BLACK);
        add(diceLabel, BorderLayout.CENTER);

        // 下方按鈕
        JButton rollButton = new JButton("擲骰子");
        rollButton.setFont(new Font("Microsoft JhengHei", Font.PLAIN, 20));

        // 按鈕事件
        rollButton.addActionListener(e -> rollDice());

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(rollButton);
        add(buttonPanel, BorderLayout.SOUTH);

        // 顯示視窗
        setVisible(true);
    }

    // 擲骰子
    private void rollDice() {

        // 產生 1~6 的隨機數
        int dice = random.nextInt(6) + 1;

        // 更新次數與總和
        count++;
        sum += dice;
```

```
        // 更新骰子點數
        diceLabel.setText(String.valueOf(dice));

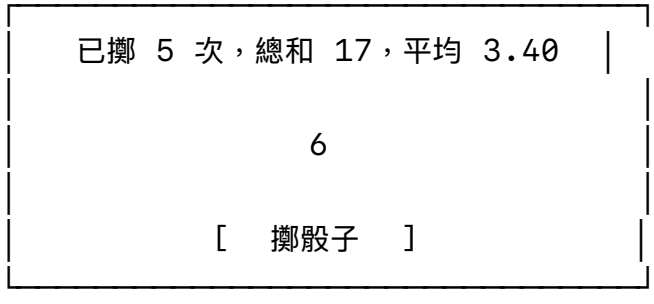
        // 依照點數改變文字顏色
        if (dice == 6) {
            diceLabel.setForeground(Color.GREEN);
        } else if (dice == 1) {
            diceLabel.setForeground(Color.RED);
        } else {
            diceLabel.setForeground(Color.BLACK);
        }

        // 計算平均
        double average = (double) sum / count;

        // 更新上方資訊
        infoLabel.setText(
            String.format("已擲 %d 次，總和 %d，平均 %.2f",
                    count, sum, average)
        );
    }

    public static void main(String[] args) {
        new DiceSimulator();
    }
}
```

執行後大概會長這樣



而且會符合：

- ✅ 標題：`骰子模擬器`
- ✅ 視窗大小：`400 × 320`

- ✅ 關閉視窗 → 程式結束
- ✅ 開啟時置中
- ✅ 中央顯示骰子點數
- ✅ 點數字體 60pt
- ✅ 「擲骰子」按鈕
- ✅ 每次隨機 1～6
- ✅ 上方顯示擲骰次數、總和、平均
- ✅ 平均保留小數點後兩位
- ✅ 6 → 綠色
- ✅ 1 → 紅色
- ✅ 2～5 → 黑色

如果你的 OOP 作業**有指定 class 名稱**（例如老師要求 A1143308_Dice），只要把 DiceSimulator 改成老師要求的名稱即可。

你說：

可不可以有模擬骰子的造型

ChatGPT 說：

你說：

對我想要像圖片這樣 可以生成程式碼給我嗎

ChatGPT 說：

可以！如果你想要的是**像剛才圖片那樣真的有「骰子造型」**，而不是只顯示數字，我可以直接用 Java Swing 的 paintComponent() 自己畫一顆骰子。

這樣**不需要另外下載圖片**，而且按「擲骰子」後，骰子上的點數也會跟著變化。

完整程式碼

```
import javax.swing.*;
import java.awt.*;
import java.awt.geom.RoundRectangle2D;
import java.util.Random;

public class DiceSimulator extends JFrame {

    private JLabel infoLabel;
```

```
    private DicePanel dicePanel;

    private int count = 0;
    private int sum = 0;

    private Random random = new Random();

    public DiceSimulator() {

        // ===== 視窗設定 =====
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null); // 視窗置中

        setLayout(new BorderLayout());

        // ===== 上方資訊 =====
        infoLabel = new JLabel(
                "已擲 0 次，總和 0，平均 0.00",
                JLabel.CENTER
        );

        infoLabel.setFont(
                new Font("Microsoft JhengHei", Font.PLAIN, 16)
        );

        add(infoLabel, BorderLayout.NORTH);


        // ===== 中間骰子 =====
        dicePanel = new DicePanel();

        add(dicePanel, BorderLayout.CENTER);


        // ===== 下方按鈕 =====
        JButton rollButton = new JButton("擲骰子");

        rollButton.setFont(
                new Font("Microsoft JhengHei", Font.BOLD, 18)
```

```
        );

        rollButton.setPreferredSize(new Dimension(120, 45));

        // 按下按鈕
        rollButton.addActionListener(e -> rollDice());

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(rollButton);

        add(buttonPanel, BorderLayout.SOUTH);

        setVisible(true);
    }


    // ===== 擲骰子 =====
    private void rollDice() {

        // 產生 1~6
        int dice = random.nextInt(6) + 1;

        // 更新資料
        count++;
        sum += dice;

        // 更新骰子
        dicePanel.setNumber(dice);

        // 計算平均
        double average = (double) sum / count;

        // 更新上方文字
        infoLabel.setText(
                String.format(
                        "已擲 %d 次，總和 %d，平均 %.2f",
                        count,
                        sum,
                        average
                )
        );
```

```
    }


    // ========================================================
    // 自訂骰子面板
    // ========================================================

    class DicePanel extends JPanel {

        private int number = 1;

        public DicePanel() {
            setBackground(Color.WHITE);
        }

        public void setNumber(int number) {
            this.number = number;
            repaint();
        }


        @Override
        protected void paintComponent(Graphics g) {

            super.paintComponent(g);

            Graphics2D g2 = (Graphics2D) g.create();

            // 開啟反鋸齒
            g2.setRenderingHint(
                    RenderingHints.KEY_ANTIALIASING,
                    RenderingHints.VALUE_ANTIALIAS_ON
            );

            // ===== 骰子大小 =====
            int size = 130;

            int x = (getWidth() - size) / 2;
            int y = (getHeight() - size) / 2;

            // ===== 骰子陰影 =====
```

```
        g2.setColor(new Color(180, 180, 180));

        g2.fillRoundRect(
                x + 6,
                y + 8,
                size,
                size,
                30,
                30
        );


        // ===== 骰子本體 =====
        g2.setColor(Color.WHITE);

        g2.fillRoundRect(
                x,
                y,
                size,
                size,
                30,
                30
        );


        // ===== 骰子邊框 =====
        g2.setColor(new Color(150, 150, 150));

        g2.setStroke(new BasicStroke(2));

        g2.drawRoundRect(
                x,
                y,
                size,
                size,
                30,
                30
        );


        // ===== 點的設定 =====
```

```
int pipSize = 20;

int left = x + 35;
int center = x + size / 2 - pipSize / 2;
int right = x + size - 55;

int top = y + 30;
int middle = y + size / 2 - pipSize / 2;
int bottom = y + size - 50;


// 黑色骰子點
g2.setColor(Color.BLACK);


// ===== 根據點數畫點 =====

switch (number) {

    case 1:
        drawPip(g2, center, middle, pipSize);
        break;

    case 2:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 3:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, center, middle, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;

    case 4:
        drawPip(g2, left, top, pipSize);
        drawPip(g2, right, top, pipSize);
        drawPip(g2, left, bottom, pipSize);
        drawPip(g2, right, bottom, pipSize);
        break;
```

```
                case 5:
                    drawPip(g2, left, top, pipSize);
                    drawPip(g2, right, top, pipSize);
                    drawPip(g2, center, middle, pipSize);
                    drawPip(g2, left, bottom, pipSize);
                    drawPip(g2, right, bottom, pipSize);
                    break;

                case 6:
                    drawPip(g2, left, top, pipSize);
                    drawPip(g2, right, top, pipSize);

                    drawPip(g2, left, middle, pipSize);
                    drawPip(g2, right, middle, pipSize);

                    drawPip(g2, left, bottom, pipSize);
                    drawPip(g2, right, bottom, pipSize);
                    break;
            }

            g2.dispose();
        }


        // ===== 畫骰子上的一個點 =====
        private void drawPip(
                Graphics2D g2,
                int x,
                int y,
                int size
        ) {

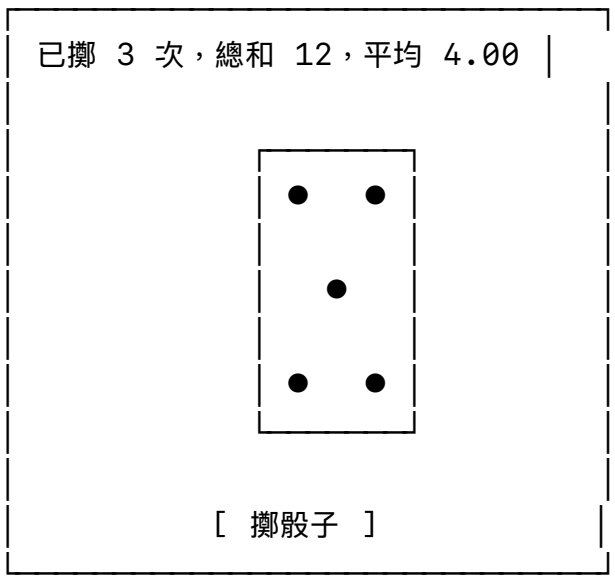
            g2.fillOval(x, y, size, size);
        }
    }


    // ===== 主程式 =====
    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
```

```
            new DiceSimulator();
        });
    }
}
```

這個版本會有



而且按下按鈕後：

- 1 → 顯示一個點
- 2 → 顯示兩個點
- 3 → 顯示三個點
- 4 → 顯示四個點
- 5 → 顯示五個點
- 6 → 顯示六個點
- 每次擲骰子都會更新「次數、總和、平均」
- 骰子是**程式自己畫出來的**，不需要圖片檔

如果你想要**更像你剛才那張圖片的「立體骰子」**，我也可以把這份程式改成有**圓角**、**陰影**、**漸層**、**3D感**的版本，而且仍然只需要一個 `.java` 檔案。